Overloaded Operators and Friend Access

C++ Interlude 5

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overloaded Operators

- To overload an operator, you define an operator method whose name has the form:

  operator*symbol*

- where symbol is the operator that you want to overload
- An overloaded operator has more than one meaning
  - In class instances, must define new meaning for operators such as the equality **operator** ==
  - For lists they are equal if they have identical lengths and items

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overloading == for Equality

```
#include <iostream>
struct Node {
  int data;
  Node* next;
  Node(int value) : data(value), next(nullptr) {} };

public:
   LinkedList() : head(nullptr) {}

   // Overloaded == operator to compare two linked lists
   bool operator==(const LinkedList& other) const {
      Node* current1 = head;
      Node* current2 = other.head;

      // Traverse both lists and compare the data of each node
      while (current1 && current2) {
         if (current1->data != current2->data) {
            return false; // Lists are not equal if any data doesn't match
         }
         current1 = current1->next;
         current2 = current2->next;
      }
   }
   // Both lists should end at the same time (same length)
   return current1 == nullptr && current2 == nullptr;
}
```

```
int main() {
   LinkedList list1;
   LinkedList list2;

   // Insert values into list1
   list1.insert(1);
   list1.insert(2);

   // Insert values into list2
   list2.insert(1);
   list2.insert(2);

   // Compare lists using overloaded == operator
   if (list1 == list2) {
      std::cout << "The lists are equal." << std::endl;
   } else {
      std::cout << "The lists are not equal." << std::endl;
   }

   return 0;
}
```

Linked list implementation of overloading ==

Overloading = for Assignment

```
// Without overloaded = operator, you end up with shallow copy
#include <iostream>
struct Node {
   int data;
   Node* next;
   Node(int value) : data(value), next(nullptr) {}
};

class LinkedList {
private:
   Node* head;

public:
// Overloaded assignment operator to perform deep copy
   LinkedList& operator=(const LinkedList& other) {
      // Self-assignment check
      if (this == &other) {
         return *this;
      }

      // First, clear the current list
      this->clear();

      // Deep copy: create a new list by copying nodes from the other list
      Node* temp = other.head;
      while (temp) {
         this->insert(temp->data);
         temp = temp->next;
      }
      return *this;
   }

};
```

```
int main() {
   LinkedList list1;
   LinkedList list2;

   // Insert values into list1
   list1.insert(1);
   list1.insert(2);

/* Use the overloaded assignment operator to
copy list1 into list2 */

   list2 = list1;

return 0;
}
```

Linked list implementation of overloading =

Overloading + for Concatenation

```
#include <iostream>

struct Node {
   int data;
   Node* next;
   Node(int value) : data(value), next(nullptr) {}
};
class LinkedList {
private:
   Node* head;

public:
// Overloaded + operator to concatenate two linked lists
  LinkedList operator+(const LinkedList& other) const {
      LinkedList result;
      Node* temp = head;
      // Insert all elements from the current list (this)
      while (temp) {
         result.insert(temp->data);
         temp = temp->next;
      }
      // Insert all elements from the other list
      temp = other.head;
      while (temp) {
         result.insert(temp->data);
         temp = temp->next;
      }
      return result; // Return the concatenated result list
   }
};
```

```
int main() {
   LinkedList list1;
   LinkedList list2;

   // Insert values into list1
   list1.insert(1);
   list1.insert(2);
   list1.insert(3);

   // Insert values into list2
   list2.insert(4);
   list2.insert(5);
   list2.insert(6);

   /* Use the overloaded + operator to concatenate list1
and list2 */
   LinkedList list3 = list1 + list2;

   // Display the result of concatenation
   std::cout << "Concatenated List (List 1 + List 2): ";
   list3.display();

   return 0;
}
```

Guidelines for Overloading Operators

- Overloading an operator is the same as writing a method whose name is
  operatorsymbol
- where symbol is the operator that you want to overload.
- Overloaded operators are typically placed in the public section of the class declaration.
- When you decide to overload an operator, be sure that the new operation closely matches how the operator is already used. If that is not possible, consider creating a method whose name is not operatorsymbol to perform the operation.
- You can overload any C++ operator except
  . .* :: ?: sizeof
- Commonly overloaded operators are the assignment (=), equality (= = and !=), and relational (<, <=, >, >=) operators.
- You cannot define new operators by overloading symbols that are not already C++ operators.
- You cannot change the standard precedence of a C++ operator.
- You cannot change the number of arguments for an overloaded operator, since they represent the operator's operands.
- At least one operand of an overloaded operator must be an instance of a class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

C++ Interlude 5

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

C++ Interlude 5

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overloaded Operators

```
template <class ItemType>
bool LinkedList<ItemType>::operator==(const
                                      LinkedList<ItemType>& rightHandSide) const
{
   bool isEqual = true; // Assume equal
   // First check whether the number of items is the same
   if (itemCount != rightHandSide.getLength())
       isEqual = false;
   else
   {  // Then compare items
      auto leftSidePtr = headPtr;
      auto rightSidePtr = rightHandSide.headPtr;

      while ((leftSidePtr != nullptr) && (rightSidePtr != nullptr) && isEqual)
```

Linked list implementation of overloading ==

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overloaded Operators

```
      auto rightSidePtr = rightHandSide.headPtr;

      while ((leftSidePtr != nullptr) && (rightSidePtr != nullptr) && isEqual)
      {
         ItemType leftItem = leftSidePtr->getItem();
         ItemType rightItem = rightSidePtr->getItem();
         isEqual = (leftItem == rightItem);

         leftSidePtr = leftSidePtr->getNext();
         rightSidePtr = rightSidePtr->getNext();
      }  // end while
   }  // end if

   return isEqual;
}  // end operator==
```

Linked list implementation of overloading ==

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overloading = for Assignment

- Without overloaded assignment operator
  - End up with shallow copy
- Implementation

```
template <class ItemType>
LinkedList<ItemType>& LinkedList<ItemType>::operator=(const
                                        LinkedList<ItemType>& rightHandSide)
{
   // Check for assignment to self
   if (this != &rightHandSide)
   {
       this->clear();                        // Deallocate left-hand side
       copyListNodes(rightHandSide);         // Copy list nodes
       itemCount = rightHandSide.itemCount;  // Copy size of list
   }   // end if

   return *this;
}  // end operator=
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Overloading + for Concatenation

- Declaration

```
LinkedList<ItemType>&
          operator+(const LinkedList<ItemType>& rightHandSide) const;
```

- Logic

```
concatList = a new, empty instance of LinkedList
leftChain = a copy of the chain of nodes in this list
rightChain = a copy of the chain of nodes in the list rightHandSide
Set the last node of leftChain to point to the first node of rightChain
concatList.headPtr = leftChain.headPtr
concatList.itemCount = itemCount + rightHandSide.itemCount
return concatList
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Friend Access and Overloading <<

- Functions and classes
  - Can be friends of a class
- We wish to grant operator<< class access to private and protected data of LinkedList

```
template<class friendItemType>
friend std::ostream& operator<<(std::ostream& outputStream,
                                const LinkedList<friendItemType>& outputList);
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Friend Access and Overloading <<

```
template<class friendItemType>
std::ostream& operator<<(std::ostream& outStream,
                        const LinkedList<friendItemType>& outputList)
{
   int position = 1;
   auto curPtr = outputList.headPtr;
   while (curPtr != nullptr)
   {
      outStream << position << "\t" << curPtr->getItem() << std::endl;
      curPtr = curPtr->getNext();
      position++;
   }  // end while

   return outStream;
}  // end operator<<
```

A friend method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Friend Access and Overloading <<

```
template<class ItemType>
class ListNode                          // A node on the list
{
private:
   ItemType item;                       // A data item on the list
   std::shared_ptr<Node<ItemType>> next; // Pointer to next node

   Node();
   Node(const ItemType& nodeItem, std::shared_ptr<Node<ItemType>> nextNode);

   // Friend class - can access private parts
   template<class friendItemType>
   friend class LinkedList<friendItemType>;
}; // end ListNode
```

A friend class

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Guidelines for Overloading Operators

- Overloading an operator is the same as writing a method whose name is

  `operator`*symbol*

  where *symbol* is the operator that you want to overload.
- Overloaded operators are typically placed in the public section of the class declaration.
- When you decide to overload an operator, be sure that the new operation closely matches how the operator is already used. If that is not possible, consider creating a method whose name is not `operator`*symbol* to perform the operation.
- Commonly overloaded operators are the assignment (=), equality (== and !=),
- At least one operand of an overloaded operator must be an instance of a class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved